



OBJECTIFS DE LA SEANCE

À la fin de cette séance, vous serez capable de :

- Comprendre l'architecture backend MVC
- Créer une API REST avec Express.js
- Connecter MongoDB avec Mongoose
- Créer des modèles de données (User, Movie, Rental)
- Initialiser la base de données avec des données de test

RESSOURCES

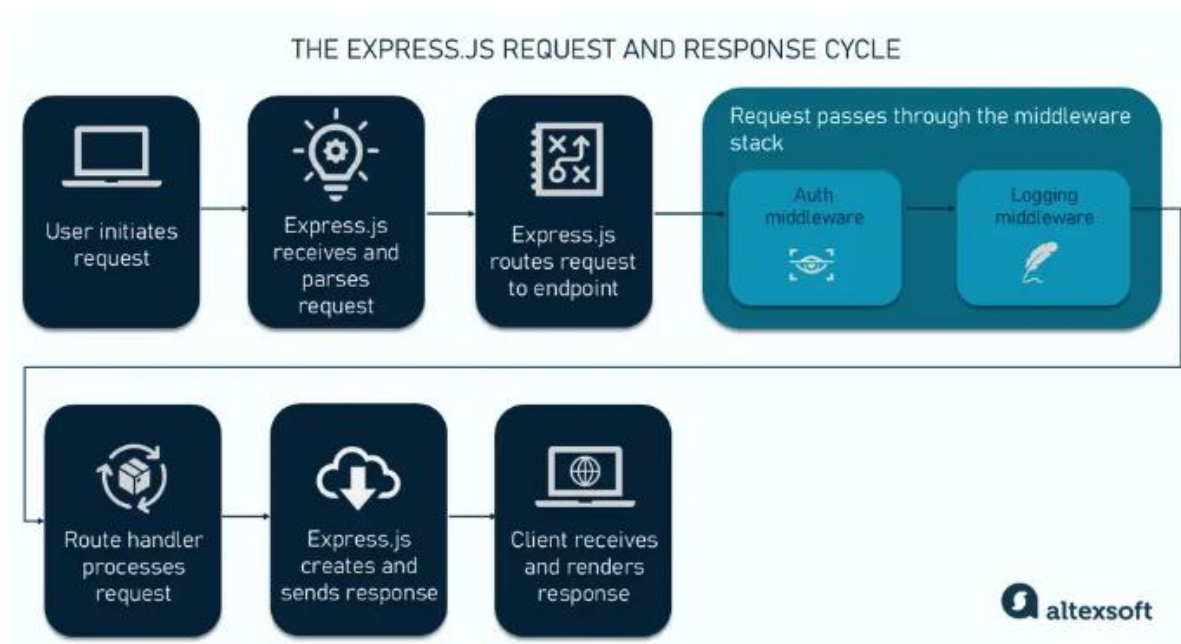
- [Mongoose Documentation](#)
- [MongoDB Manual](#)
- [Express.js Guide](#)

PREREQUIS

- Node.js installé
- Connaissance basique de JavaScript
- Séances 1 à 6 terminées

Normalement au TP01 vous avez déjà installé les modules nécessaires, si ce n'est pas le cas installez :

Module	Utilité
bcryptjs	Sécurise les mots de passe en les hachant avant de les stocker en base de données.
cors	Autorise ou restreint les requêtes provenant de domaines différents de celui du serveur.
dotenv	Charge les variables d'environnement (clés secrètes, ports) depuis un fichier .env vers process.env.
express	Framework minimaliste qui facilite la création de routes et la gestion des requêtes HTTP.
jsonwebtoken	Génère et vérifie des jetons (tokens) sécurisés pour l'authentification des utilisateurs.
mongoose	Bibliothèque (ODM) qui simplifie les interactions avec MongoDB en utilisant des schémas.
nodemon	Relance automatiquement le serveur à chaque fois que vous modifiez et sauvegardez un fichier.

 PARTIE 1 : ARCHITECTURE BACKEND (15 MIN)**1.1 PATTERN MVC (MODEL-VIEW-CONTROLLER)**



1.2 STRUCTURE DU BACKEND

```
backend/
├── src/
│   ├── config/ # Configuration (DB, env)
│   │   └── database.js
│   ├── models/ # Schémas Mongoose
│   │   ├── User.js
│   │   ├── Movie.js
│   │   └── Rental.js
│   ├── routes/ # Définition des routes
│   │   ├── auth.routes.js
│   │   ├── movie.routes.js
│   │   └── rental.routes.js
│   ├── controllers/ # Logique des routes
│   │   ├── auth.controller.js
│   │   ├── movie.controller.js
│   │   └── rental.controller.js
│   ├── middleware/ # Middlewares personnalisés
│   │   ├── auth.middleware.js
│   │   └── error.middleware.js
│   ├── utils/ # Utilitaires
│   │   ├── validators.js
│   │   └── seed.js
│   └── server.js # Point d'entrée
├── .env # Variables d'environnement
└── package.json
```



1.3 PRINCIPES REST

Méthode	Route	Action (Controller)	Description
GET	/api/movies	index	Récupère la liste complète des films.
GET	/api/movies/:id	show	Récupère les détails d'un film spécifique via son ID.
POST	/api/movies	create	Ajoute un nouveau film dans la base MongoDB.
PUT	/api/movies/:id	update	Met à jour l'intégralité des infos d'un film existant.
DELETE	/api/movies/:id	destroy	Supprime définitivement un film de la base de données.



PARTIE 2 : CONFIGURATION MONGODB (25 MIN)

2.1 INSTALLATION DE MONGODB

OPTION A : MONGODB ATLAS (CLOUD - RECOMMANDE POUR DEBUTER)

- Créer un compte sur <https://www.mongodb.com/atlas>
- Créer un cluster gratuit (M0)
- Créer un utilisateur de base de données
- Whitelist votre IP (ou 0.0.0.0/0 pour tout autoriser)
- Obtenir la connection string

OPTION B : MONGODB LOCAL

MACOS (HOMEBREW)

```
brew tap mongodb/brew  
brew install mongodb-community  
brew services start mongodb-community
```

UBUNTU/DEBIAN

```
wget -qO - https://www.mongodb.org/static/pgp/server-6.0.asc | sudo  
apt-key add -  
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu  
focal/mongodb-org/6.0 multiverse" | sudo tee  
/etc/apt/sources.list.d/mongodb-org-6.0.list  
sudo apt-get update  
sudo apt-get install -y mongodb-org  
sudo systemctl start mongod
```

WINDOWS

- Téléchargez depuis
<https://www.mongodb.com/try/download/community>



- Installez et démarrez MongoDB Service

2.2 CONFIGURATION DE L'ENVIRONNEMENT

- Créez votre fichier .env dans backend/ :

```
# Server
PORT=5000
NODE_ENV=development

# MongoDB
MONGODB_URI=mongodb://localhost:27017/netflix
# Ou pour Atlas:
#
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/netflix?retryWrites=true&w=majority

# JWT (sera utilisé plus tard)
JWT_SECRET=votre_secret_super_securise_changez_moi_absolument
JWT_EXPIRE=7d

# Client URL (pour CORS)
CLIENT_URL=http://localhost:3000
```

- Adaptez les ports à votre environnement



2.3 CONFIGURATION DE LA CONNEXION

- Créez le fichier src/config/database.js :

```
import mongoose from 'mongoose';

const connectDB = async () => {
  try {
    const options = {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    };

    const conn = await mongoose.connect(process.env.MONGODB_URI, options);

    console.log('✅ MongoDB Connected: ${conn.connection.host}');
    console.log('📊 Database: ${conn.connection.name}');

    // Événements de connexion
    mongoose.connection.on('error', (err) => {
      console.error('❌ MongoDB connection error: ${err}');
    });

    mongoose.connection.on('disconnected', () => {
      console.log('⚠️ MongoDB disconnected');
    });

    // Gestion de l'arrêt propre
    process.on('SIGINT', async () => {
      await mongoose.connection.close();
      console.log('MongoDB connection closed through app termination');
      process.exit(0);
    });

  } catch (error) {
    console.error('❌ Error: ${error.message}');
    process.exit(1);
  }
};

export default connectDB;
```



PARTIE 3 : MODELES MONGOOSE (40 MIN)

3.1 MODELE USER

SRC/MODELS/USER.JS :

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Le nom est requis'],
    trim: true,
    minlength: [2, 'Le nom doit contenir au moins 2 caractères'],
    maxlength: [50, 'Le nom ne peut pas dépasser 50 caractères']
  },
  email: {
    type: String,
    required: [true, 'L\'email est requis'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Email invalide']
  },
  password: {
    type: String,
    required: [true, 'Le mot de passe est requis'],
    minlength: [6, 'Le mot de passe doit contenir au moins 6 caractères'],
    select: false // Ne pas retourner le password par défaut dans les queries
  },
  role: {
    type: String,
    enum: ['user', 'admin'],
    default: 'user'
  },
  avatar: {
    type: String,
    default: function() {
```




```

    return `https://ui-
avatars.com/api/?name=${encodeURIComponent(this.name)}&background=e50914&color=fff`;
  },
  isActive: {
    type: Boolean,
    default: true
  }
}, {
  timestamps: true // Ajoute createdAt et updatedAt automatiquement
});

// INDEX pour améliorer les performances
//userSchema.index({ email: 1 }); // L'index sur email est déjà créé automatiquement grâce à unique: true

// MIDDLEWARE pre-save: Hasher le mot de passe avant sauvegarde
userSchema.pre('save', async function() {
  // Ne hasher que si le password a été modifié
  if (!this.isModified('password')) {
    return
  }

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// MÉTHODE pour comparer les mots de passe
userSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

// MÉTHODE pour obtenir les infos publiques (sans password)
userSchema.methods.toJSON = function() {
  const user = this.toObject();
  delete user.password;
  return user;
};

// MÉTHODE statique pour trouver par email
userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email: email.toLowerCase() });
};

```



```
};
```

```
const User = mongoose.model('User', userSchema);
```

```
export default User;
```

Concepts clés :

- **Schema** : Définit la structure des documents
- **Validators** : Validation des données (required, min, max, match)
- **Middleware** : Code exécuté avant/après certaines opérations
- **Methods** : Fonctions sur les instances de documents
- **Statics** : Fonctions sur le modèle lui-même
- **Indexes** : Optimisation des requêtes

1. LES STATICS (METHODES DE CLASSE)

Les **Statics** sont des fonctions que vous appelez sur le **Modèle lui-même**. C'est un peu comme une fonction utilitaire pour gérer toute la collection.

- **Usage** : Rechercher des documents, statistiques globales, filtrages complexes.
- **Mot-clé** : this fait référence au **Modèle**.

2. LES METHODS (METHODES D'INSTANCE)

Les **Methods** sont des fonctions que vous appelez sur un **Document spécifique** (une instance). Elles concernent les données d'un seul objet précis.

- **Usage** : Formater le nom, vérifier un mot de passe, calculer une remise sur un produit précis.
- **Mot-clé** : this fait référence au **Document actuel**.

3. LES VALIDATORS



Pour compléter votre schéma et assurer une intégrité totale des données (surtout avec une liste de genres), la **validation** est l'étape cruciale.

Dans Mongoose, il existe deux types de validations que vous pouvez combiner :

1. VALIDATION INTEGREE (BUILT-IN)

C'est ce que nous avons déjà commencé à utiliser avec `required`, `enum` et `match`. Mongoose vérifie automatiquement que chaque élément ajouté au tableau correspond à la liste autorisée.

2. VALIDATION PERSONNALISEE (CUSTOM VALIDATOR)

C'est ici que l'on peut repousser les limites. Par exemple, vous voulez peut-être qu'un film ait **au moins un genre** mais **pas plus de trois**, et qu'il n'y ait **pas de doublons**.

```
validate: [  
  {  
    validator: function(val) {  
      // Vérifie que le tableau n'est pas vide  
      return val.length > 0;  
    },  
    message: "Un film doit avoir au moins un genre."  
  },  
  {  
    validator: function(val) {  
      // Vérifie qu'il n'y a pas plus de 5 genres  
      return val.length <= 5;  
    },  
    message: "Un film ne peut pas avoir plus de 5 genres."  
  },  
  {  
    validator: function(val) {  
      // Vérifie les doublons (Set élimine les doublons, on compare les tailles)  
      return new Set(val).size === val.length;  
    },  
    message: "La liste des genres contient des doublons."  
  }  
],
```



3.2 MODELE MOVIE

- Créez le modèle Movie

```
import mongoose from "mongoose";

const movieSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, "Le titre est requis"],
      trim: true,
      maxlength: [200, "Le titre ne peut pas dépasser 200 caractères"],
    },
    description: {
      type: String,
      required: [true, "La description est requise"],
      maxlength: [2000, "La description ne peut pas dépasser 2000 caractères"],
    },
    poster: {
      type: String,
      required: [true, "L'affiche est requise"],
      validate: {
        validator: function (v) {
          return /^https?:VV.+/.test(v);
        },
        message: "L'URL de l'affiche doit être valide",
      },
    },
    backdrop: {
      type: String,
      required: [true, "L'image de fond est requise"],
      validate: {
        validator: function (v) {
          return /^https?:VV.+/.test(v);
        },
        message: "L'URL de l'image de fond doit être valide",
      },
    },
    genre: {
      type: [String],
    },
  }
);
```



```
required: [true, "Le genre est requis"],
validate: {
  validator: function (v) {
    return v && v.length > 0;
  },
  message: "La liste des genres ne peut pas être vide.",
},
enum: {
  values: [
    "Action",
    "Comédie",
    "Drame",
    "Science-Fiction",
    "Horreur",
    "Thriller",
    "Romance",
    "Animation",
    "Documentaire",
  ],
  message: "{VALUE} n'est pas un genre valide",
},
year: {
  type: Number,
  required: [true, "L'année est requise"],
  min: [1900, "L'année doit être supérieure à 1900"],
  max: [
    new Date().getFullYear() + 2,
    "L'année ne peut pas être dans un futur lointain",
  ],
},
duration: {
  type: Number,
  required: [true, "La durée est requise"],
  min: [1, "La durée doit être positive"],
  validate: {
    validator: function (v) {
      // La durée doit être entre 1 et 500 minutes
      return v > 0 && v <= 500;
    },
    message: "La durée doit être entre 1 et 500 minutes",
  },
}
```



```

    },
  },
  price: {
    type: Number,
    required: [true, "Le prix est requis"],
    min: [0, "Le prix doit être positif"],
    default: 3.99,
    validate: {
      validator: function (v) {
        // Le prix doit avoir maximum 2 décimales
        return /^\\d+(\\.\\d{1,2})?$/\\.test(v.toString());
      },
      message: "Le prix doit avoir au maximum 2 décimales",
    },
  },
  rating: {
    type: Number,
    min: [0, "La note doit être entre 0 et 10"],
    max: [10, "La note doit être entre 0 et 10"],
    default: 0,
  },
  isAvailable: {
    type: Boolean,
    default: true,
  },
  rentalCount: {
    type: Number,
    default: 0,
    min: 0,
  },
},
{
  timestamps: true,
},
);

// INDEX pour la recherche
movieSchema.index({ title: "text", description: "text" });
movieSchema.index({ genre: 1 });
movieSchema.index({ year: -1 });
movieSchema.index({ rating: -1 });

```



```
// VIRTUAL pour la durée formatée
movieSchema.virtual("durationFormatted").get(function () {
  const hours = Math.floor(this.duration / 60);
  const minutes = this.duration % 60;
  return `${hours}h${minutes > 0 ? ` ${minutes}min` : ''}`;
});

// MÉTHODE pour incrémenter le compteur de location
movieSchema.methods.incrementRentalCount = async function () {
  this.rentalCount += 1;
  return await this.save();
};

// MÉTHODE STATIQUE pour obtenir les films récents
movieSchema.statics.getRecentMovies = function (limit = 10) {
  return this.find({ isAvailable: true }).sort({ year: -1 }).limit(limit);
};

// MÉTHODE STATIQUE pour obtenir les films populaires
movieSchema.statics.getPopularMovies = function (limit = 10) {
  return this.find({ isAvailable: true })
    .sort({ rentalCount: -1, rating: -1 })
    .limit(limit);
};

// MÉTHODE STATIQUE pour rechercher des films
movieSchema.statics.search = function (query) {
  return this.find({
    $text: { $search: query },
    isAvailable: true,
  }).sort({ score: { $meta: "textScore" } });
};

// Obtenir les films par genre
movieSchema.statics.getByGenre = function (genre) {
  return this.find({
    genre,
    isAvailable: true,
  }).sort({ rating: -1 });
};
```



```
// Obtenir les films dans une fourchette de prix
movieSchema.statics.getByPriceRange = function (minPrice, maxPrice) {
  return this.find({
    price: { $gte: minPrice, $lte: maxPrice },
    isAvailable: true,
  }).sort({ price: 1 });
};

// Obtenir les statistiques par genre
movieSchema.statics.getStatsByGenre = async function () {
  return await this.aggregate([
    {
      $match: { isAvailable: true },
    },
    {
      $group: {
        _id: "$genre",
        count: { $sum: 1 },
        avgPrice: { $avg: "$price" },
        avgRating: { $avg: "$rating" },
        totalRentals: { $sum: "$rentalCount" },
      },
    },
    {
      $sort: { count: -1 },
    },
  ]);
};

const Movie = mongoose.model("Movie", movieSchema);

export default Movie;
```

Concepts avancés :

- Notez que le genre est un tableau de chaîne, et chaque valeur sera vérifiée, attention à la casse



3.3 MODELE RENTAL

- Créez le modèle Rental

```
import mongoose from 'mongoose';

const rentalSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  movie: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Movie',
    required: true
  },
  rentalDate: {
    type: Date,
    default: Date.now
  },
  expiryDate: {
    type: Date,
    required: true,
    default: function() {
      const date = new Date();
      date.setDate(date.getDate() + 7); // 7 jours par défaut
      return date;
    }
  },
  price: {
    type: Number,
    required: true,
    min: 0
  },
  status: {
    type: String,
    enum: ['active', 'expired', 'cancelled'],
    default: 'active'
  }
})
```



```

}, {
  timestamps: true
});

// INDEX composé pour éviter les doublons et optimiser les requêtes
rentalSchema.index({ user: 1, movie: 1 });
rentalSchema.index({ status: 1, expiryDate: 1 });

// VIRTUAL pour vérifier si la location est toujours valide
rentalSchema.virtual('isValid').get(function() {
  return this.status === 'active' && new Date() < this.expiryDate;
});

// VIRTUAL pour obtenir les jours restants
rentalSchema.virtual('daysLeft').get(function() {
  if (this.status !== 'active') return 0;
  const diff = this.expiryDate - new Date();
  return Math.ceil(diff / (1000 * 60 * 60 * 24));
});

// MÉTHODE pour vérifier si la location est active
rentalSchema.methods.isActive = function() {
  return this.status === 'active' && new Date() < this.expiryDate;
};

// MÉTHODE pour marquer comme expiré
rentalSchema.methods.markAsExpired = async function() {
  this.status = 'expired';
  return await this.save();
};

// MÉTHODE STATIQUE pour obtenir les locations actives d'un utilisateur
rentalSchema.statics.getActiveRentals = function(userId) {
  return this.find({
    user: userId,
    status: 'active',
    expiryDate: { $gt: new Date() }
  })
  .populate('movie')
  .sort({ rentalDate: -1 });
};

```



```
// MÉTHODE STATIQUE pour obtenir les locations expirées
rentalSchema.statics.getExpiredRentals = function(userId) {
  return this.find({
    user: userId,
    $or: [
      { status: 'expired' },
      { expiryDate: { $lt: new Date() } }
    ]
  })
  .populate('movie')
  .sort({ expiryDate: -1 });
};

// MIDDLEWARE pre-find: Mettre à jour le statut des locations expirées
rentalSchema.pre(/^find/, async function() {
  // Marquer comme expirées les locations dont la date est dépassée
  await this.model.updateMany(
    {
      expiryDate: { $lt: new Date() },
      status: 'active'
    },
    { status: 'expired' }
  );
});

const Rental = mongoose.model('Rental', rentalSchema);

export default Rental;
```



PARTIE 4 : SCRIPT DE SEED (20 MIN)

4.1 CREEZ LE SCRIPT DE SEED

- Créez et adaptez le fichier src/utils/seed.js :

```
import mongoose from 'mongoose';
import dotenv from 'dotenv';
import { fileURLToPath } from 'url';
import { dirname, join } from 'path';
import { readFileSync } from 'fs';
import connectDB from '../config/database.js';
import User from '../models/User.js';
import Movie from '../models/Movie.js';
import Rental from '../models/Rental.js';

// Configuration pour ES modules
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);

// Charger les variables d'environnement
dotenv.config({ path: join(__dirname, '../.env') });

// Charger les données de films
const moviesPath = join(__dirname, '../data/movies.json');
const moviesData = JSON.parse(readFileSync(moviesPath, 'utf-8'));

const seedDatabase = async () => {
  try {
    // Connexion à la base de données
    await connectDB();

    console.log('🧹 Nettoyage de la base de données...');

    // Supprimer toutes les données existantes
    await User.deleteMany({});
    await Movie.deleteMany({});
    await Rental.deleteMany({});

    console.log('✅ Base de données nettoyée');
```

R4.10 : Complément web



TP 2026 #07 : Backend -
Express/MongoDB

```
// Créer un utilisateur admin
console.log('👤 Création de l\'utilisateur admin...');
const admin = await User.create({
  name: 'Admin Netflix',
  email: 'admin@netflix.com',
  password: 'admin123',
  role: 'admin'
});
console.log('✅ Admin créé: ${admin.email}');

// Créer des utilisateurs de test
console.log('👥 Création des utilisateurs de test...');
const users = await User.create([
  {
    name: 'John Doe',
    email: 'john@example.com',
    password: 'password123'
  },
  {
    name: 'Jane Smith',
    email: 'jane@example.com',
    password: 'password123'
  },
  {
    name: 'Bob Martin',
    email: 'bob@example.com',
    password: 'password123'
  }
]);
console.log('✅ ${users.length} utilisateurs créés');

// Insérer les films
console.log('🎬 Insertion des films...');
const movies = await Movie.insertMany(moviesData);
console.log('✅ ${movies.length} films insérés');

// Créer quelques locations de test
console.log('📄 Création de locations de test...');
const rentals = await Rental.create([
  {
```



```

    user: users[0]._id,
    movie: movies[0]._id,
    price: movies[0].price,
    rentalDate: new Date(),
    expiryDate: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)
  },
  {
    user: users[0]._id,
    movie: movies[1]._id,
    price: movies[1].price,
    rentalDate: new Date(),
    expiryDate: new Date(Date.now() + 5 * 24 * 60 * 60 * 1000)
  },
  {
    user: users[1]._id,
    movie: movies[2]._id,
    price: movies[2].price,
    rentalDate: new Date(Date.now() - 10 * 24 * 60 * 60 * 1000),
    expiryDate: new Date(Date.now() - 3 * 24 * 60 * 60 * 1000),
    status: 'expired'
  }
]);
console.log('✅ ${rentals.length} locations créées');

// Mettre à jour le compteur de location des films
await movies[0].incrementRentalCount();
await movies[1].incrementRentalCount();
await movies[2].incrementRentalCount();

console.log('\n🎉 Base de données initialisée avec succès!');
console.log('\n📊 Résumé:');
console.log(` - Admin: admin@netflix.com / admin123`);
console.log(` - Users: ${users.length}`);
console.log(` - Movies: ${movies.length}`);
console.log(` - Rentals: ${rentals.length}`);

process.exit(0);

} catch (error) {
  console.error('❌ Erreur lors du seed: ${error.message}');
  console.error(error);
}

```



```
    process.exit(1);  
  }  
};  
// Exécuter le seed  
seedDatabase();
```

4.2 VERIFIEZ LE PACKAGE.JSON

Normalement le script du seed est déjà intégré au package.json. Si ce n'est pas le cas ajoutez-le.

```
"scripts": {  
  "dev": "nodemon src/server.js",  
  "start": "node src/server.js",  
  "seed": "node src/utlis/seed.js"  
},
```

4.3 EXECUTEZ LE SEED

```
cd backend  
npm run seed
```

Resultat attendu :



```
✓ MongoDB Connected: ac-6zxtpc7-shard-00-00.famu7tj.mongodb.net
Database: netflix
Nettoyage de la base de données...
✓ Base de données nettoyée
Création de l'utilisateur admin...
✓ Admin créé: admin@netflix.com
Création des utilisateurs de test...
✓ 3 utilisateurs créés
Insertion des films...
✓ 5 films insérés
Création de locations de test...
✓ 3 locations créées
```

🎉 Base de données initialisée avec succès!

```
Résumé:
- Admin: admin@netflix.com / admin123
- Users: 3
- Movies: 5
- Rentals: 3
```




Documents 4 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#) ↗

[+ ADD DATA](#)
[UPDATE](#)
[DELETE](#)
[EXPORT DATA](#)
[EXPORT CODE](#)
100

```

_id: ObjectId('6992cfee80fd80cb182f4f95')
name: "Admin Netflix"
email: "admin@netflix.com"
password: "$2b$10$f3s49LL0uS7q93xPxcgYq0hSDpzML1KwvstAcx960oiuiy99oBUWq"
role: "admin"
isActive: true
avatar: "https://ui-avatars.com/api/?name=Admin%20Netflix&background=e50914&col..."
createdAt: 2026-02-16T08:06:06.253+00:00
updatedAt: 2026-02-16T08:06:06.253+00:00
__v: 0

```

```

_id: ObjectId('6992cfee80fd80cb182f4f97')
name: "John Doe"
email: "john@example.com"
password: "$2b$10$FCr0STwMuhTDttmpwgqi40LvepT7Hg9toJGRHi967DKw5u5/.Il2e"
role: "user"
isActive: true
avatar: "https://ui-avatars.com/api/?name=John%20Doe&background=e50914&color=ff..."
createdAt: 2026-02-16T08:06:06.372+00:00
updatedAt: 2026-02-16T08:06:06.372+00:00
__v: 0

```

```

_id: ObjectId('6992cfee80fd80cb182f4f99')
name: "Bob Martin"
email: "bob@example.com"
password: "$2b$10$rB2LArD4JbA..vJ0tz/co.qwNQsYf2g7VuS0U5lqtUhdhTaJhD8/C"

```



PARTIE 5 : SERVEUR EXPRESS DE BASE (20 MIN)

5.1 CREEZ LE SERVEUR

- Créez le fichier src/server.js :

```
import express from 'express';
import dotenv from 'dotenv';
import cors from 'cors';
import connectDB from './config/database.js';
import mongoose from "mongoose";

// Charger les variables d'environnement
dotenv.config();

// Initialiser Express
const app = express();
const PORT = process.env.PORT || 5000;

// Connecter à MongoDB
connectDB();

// Middlewares globaux
app.use(cors({
  origin: process.env.CLIENT_URL || 'http://localhost:3000',
  credentials: true
}));

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Logger simple pour le développement
if (process.env.NODE_ENV === 'development') {
  app.use((req, res, next) => {
    console.log(`${req.method} ${req.path}`);
    next();
  });
}
```



```
// Routes de test
app.get('/', (req, res) => {
  res.json({
    message: 'Netflix API',
    version: '1.0.0',
    endpoints: {
      health: '/api/health',
      movies: '/api/movies',
      auth: '/api/auth',
      rentals: '/api/rentals'
    }
  });
});

app.get('/api/health', (req, res) => {
  res.json({
    status: 'OK',
    message: 'API Netflix is running',
    timestamp: new Date().toISOString(),
    database: mongoose.connection.readyState === 1 ? 'connected' : 'disconnected'
  });
});

// TODO: Importer et utiliser les routes - Prochaine séance si vous n'êtes pas trop lent ☺
// import movieRoutes from './routes/movie.routes.js';
// import authRoutes from './routes/auth.routes.js';
// import rentalRoutes from './routes/rental.routes.js';
// app.use('/api/movies', movieRoutes);
// app.use('/api/auth', authRoutes);
// app.use('/api/rentals', rentalRoutes);

// Gestion des erreurs 404
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: 'Route not found',
    path: req.path
  });
});
```



```
// Middleware de gestion d'erreurs global
app.use((err, req, res, next) => {
  console.error('Error:', err);

  // Erreur de validation Mongoose
  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map(e => e.message);
    return res.status(400).json({
      success: false,
      message: 'Erreur de validation',
      errors: messages
    });
  }

  // Erreur de cast Mongoose (ID invalide)
  if (err.name === 'CastError') {
    return res.status(400).json({
      success: false,
      message: 'ID invalide'
    });
  }

  // Erreur de duplication (email déjà utilisé)
  if (err.code === 11000) {
    const field = Object.keys(err.keyPattern)[0];
    return res.status(400).json({
      success: false,
      message: `${field} déjà utilisé`
    });
  }

  // Erreur par défaut
  res.status(err.statusCode || 500).json({
    success: false,
    message: err.message || 'Internal Server Error',
    ...(process.env.NODE_ENV === 'development' && {
      stack: err.stack,
      error: err
    })
  });
});
```

R4.10 : Complément web



TP 2026 #07 : Backend -
Express/MongoDB

```
});
```

```
// Démarrer le serveur
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
  console.log(`📄 Environment: ${process.env.NODE_ENV}`);
  console.log(`🌐 API URL: http://localhost:${PORT}/api`);
});
```

```
// Gestion des erreurs non gérées
process.on('unhandledRejection', (err) => {
  console.error('❌ Unhandled Rejection:', err);
  process.exit(1);
});
```

```
process.on('uncaughtException', (err) => {
  console.error('❌ Uncaught Exception:', err);
  process.exit(1);
});
```

```
export default app;
```



5.2 TESTEZ LE SERVEUR

npm run dev

```

Server running on http://localhost:4500
Environment: development
API URL: http://localhost:4500/api
MongoDB Connected: ac-6zxtpc7-shard-00-00.famu7tj.mongodb.net
Database: netflix
  
```

- Testez dans le navigateur

http://localhost:5000 → Informations API	http://localhost:5000/api/health → Health check
<pre> { message: "Netflix API", version: "1.0.0", endpoints: { health: "/api/health", movies: "/api/movies", auth: "/api/auth", rentals: "/api/rentals" } } </pre>	<pre> { status: "OK", message: "API Netflix is running", timestamp: "2026-02-16T08:19:37.922Z", database: "connected" } </pre>

EXERCICE 1 : TESTER LES MODELES (20 MIN)

OBJECTIF

Créer un script de test pour valider les modèles Mongoose.

INSTRUCTIONS

- Créez le fichier `src/Utils/testModels.js` :

```

import mongoose from 'mongoose';
import dotenv from 'dotenv';
import connectDB from '../config/database.js';
import User from '../models/User.js';
  
```



```
import Movie from '../models/Movie.js';
import Rental from '../models/Rental.js';

dotenv.config();

const testModels = async () => {
  try {
    await connectDB();

    console.log('🔧 Tests des modèles...\n');

    // Test 1: Créer un utilisateur
    console.log('Test 1: Création d\'un utilisateur');
    const testUser = await User.create({
      name: 'Test User',
      email: 'test@test.com',
      password: 'test123'
    });
    console.log('✅ Utilisateur créé:', testUser.toJSON());
    console.log('Avatar auto-généré:', testUser.avatar);

    // Test 2: Tester la méthode comparePassword
    console.log('\nTest 2: Comparaison de mot de passe');
    const userWithPassword = await User.findById(testUser._id).select('+password');
    const isMatch = await userWithPassword.comparePassword('test123');
    console.log('✅ Password match:', isMatch);

    // Test 3: Créer un film
    console.log('\nTest 3: Création d\'un film');
    const testMovie = await Movie.create({
      title: 'Test Movie',
      description: 'Un film de test',
      poster: 'https://example.com/poster.jpg',
      backdrop: 'https://example.com/backdrop.jpg',
      genre: 'Action',
      year: 2024,
      duration: 120,
      price: 4.99,
      rating: 7.5
    });
  }
};
```



```
console.log('✅ Film créé:', testMovie.title);
console.log('  Durée formatée:', testMovie.durationFormatted);

// Test 4: Créer une location
console.log('\nTest 4: Création d\'une location');
const testRental = await Rental.create({
  user: testUser._id,
  movie: testMovie._id,
  price: testMovie.price
});
console.log('✅ Location créée');
console.log('  Jours restants:', testRental.daysLeft);
console.log('  Est active:', testRental.isActive());

// Test 5: Populate
console.log('\nTest 5: Populate (relations)');
const rentalWithDetails = await Rental.findById(testRental._id)
  .populate('user', 'name email')
  .populate('movie', 'title price');
console.log('✅ Location avec détails:', {
  user: rentalWithDetails.user.name,
  movie: rentalWithDetails.movie.title,
  price: rentalWithDetails.price
});

// Test 6: Méthodes statiques
console.log('\nTest 6: Méthodes statiques');
const activeRentals = await Rental.getActiveRentals(testUser._id);
console.log('✅ Locations actives:', activeRentals.length);

// Nettoyage
console.log('\n🧹 Nettoyage...');
await User.deleteOne({ _id: testUser._id });
await Movie.deleteOne({ _id: testMovie._id });
await Rental.deleteOne({ _id: testRental._id });

console.log('\n🎉 Tous les tests sont passés!');
process.exit(0);

} catch (error) {
  console.error('❌ Erreur:', error);
}
```




```

    process.exit(1);
  }
};

testModels();

```

⚠ Attention : Dans mongoDB l'identifiant est noté **_id**

- Exécuter :

node src/utils/testModels.js

```

● david@MDJ-Studio backend % node src/utils/testModels.js
[dotenv@17.2.3] injecting env (6) from .env -- tip: 📦 prevent building .env in docker: https://dotenvx.com/prebuild
✅ MongoDB Connected: ac-6zxtpc7-shard-00-01.famu7tj.mongodb.net
🇫🇷 Database: netflix
🚀 Tests des modèles...

Test 1: Création d'un utilisateur
✅ Utilisateur créé: {
  name: 'Test User',
  email: 'test@test.com',
  role: 'user',
  isActive: true,
  _id: new ObjectId('6992d79005e5ff2f41bea3ab'),
  avatar: 'https://ui-avatars.com/api/?name=Test%20User&background=e50914&color=fff',
  createdAt: 2026-02-16T08:38:40.247Z,
  updatedAt: 2026-02-16T08:38:40.247Z,
  __v: 0
}
Avatar auto-généré: https://ui-avatars.com/api/?name=Test%20User&background=e50914&color=fff

Test 2: Comparaison de mot de passe
✅ Password match: true

Test 3: Création d'un film
✅ Film créé: Test Movie
Durée formatée: 2h

Test 4: Création d'une location
✅ Location créée
  Jours restants: 7
  Est active: true

Test 5: Populate (relations)
✅ Location avec détails: { user: 'Test User', movie: 'Test Movie', price: 4.99 }

Test 6: Méthodes statiques
✅ Locations actives: 1

🧹 Nettoyage...

🎉 Tous les tests sont passés!

```



EXERCICE 2 : AJOUTER DES VALIDATIONS PERSONNALISEES (15 MIN)

OBJECTIF

Améliorer le modèle Movie avec des validations supplémentaires.

INSTRUCTIONS

- Dans `models/Movie.js`, ajoutez les validateurs pour vérifier que:
 - o La durée soit comprise entre 1 et 500 minutes
 - o Que le film n'a pas plus de 5 genres
 - o Le prix contienne au moins 2 décimales
- Ajoutez dans les tests la vérification du prix:

```
// Dans testModels.js, ajouter un test d'erreur
try {
  await Movie.create({
    title: 'Film invalide',
    duration: 600, // Trop long
    price: 3.999 // Trop de décimales
  });
} catch (error) {
  console.log('✅ Validation échouée comme prévu:', error.message);
}
```

EXERCICE 3 : METHODES DE REQUETE AVANCEES (20 MIN)

OBJECTIF

Ajouter des méthodes statiques utiles au modèle Movie.

INSTRUCTIONS

- Ajoutez dans `models/Movie.js` les méthodes statiques

Nom	Résultat
getByGenre	les films par genre find



getByPriceRange	les films dans une fourchette de prix	find
getStatsByGenre	Statistiques par genre	aggregate [match, group, sort]

- Ajoutez les tests

```
const sciFiMovies = await Movie.getByGenre("Science-Fiction");
console.log("Films Sci-Fi:", sciFiMovies.length);
```

```
const affordableMovies = await Movie.getByPriceRange(0, 4);
console.log("Films à moins de 4€:", affordableMovies.length);
```

```
const stats = await Movie.getStatsByGenre();
console.log("Statistiques par genre:", stats);
```

```
Films Sci-Fi: 2
Films à moins de 4€: 4
Statistiques par genre: [
  {
    _id: 'Science-Fiction',
    count: 2,
    avgPrice: 4.49,
    avgRating: 8.7,
    totalRentals: 2
  },
  {
    _id: 'Thriller',
    count: 1,
    avgPrice: 3.99,
    avgRating: 8.9,
    totalRentals: 0
  },
  {
    _id: 'Action',
    count: 1,
    avgPrice: 3.99,
    avgRating: 9,
    totalRentals: 1
  },
  {
    _id: 'Drame',
    count: 1,
    avgPrice: 3.99,
    avgRating: 8.8,
    totalRentals: 0
  }
]
```



CHALLENGE BONUS : MODELE REVIEW

OBJECTIF

Créer un modèle Review pour permettre aux utilisateurs de noter et commenter les films.

FONCTIONNALITES

- Relation avec User et Movie
- Note (1-5 étoiles)
- Commentaire
- Un seul review par utilisateur par film
- Calcul de la note moyenne d'un film

CHECKLIST DE FIN DE SEANCE

- MongoDB installé et connecté
- Les 3 modèles créés (User, Movie, Rental)
- Script de seed fonctionnel
- Base de données initialisée avec données de test
- Serveur Express démarre sans erreur
- Route `/api/health` fonctionne
- Au moins 1 exercice terminé
- Code committé



À RETENIR

MONGOOSE CONCEPTS

- **Schema** : Définit la structure des documents
- **Model** : Constructeur pour créer et lire des documents
- **Document** : Instance d'un modèle
- **Validators** : Validation des données avant sauvegarde
- **Middleware** : Fonctions exécutées à certains moments du cycle de vie
- **Virtuals** : Propriétés calculées non stockées en base
- **Statics** : Méthodes sur le modèle (Model.method())
- **Methods** : Méthodes sur les documents (document.method())
- **Populate** : Résoudre les références entre documents

TYPES DE DONNEES MONGOOSE

String	// Chaîne de caractères
Number	// Nombre
Boolean	// Booléen
Date	// Date
ObjectId	// Référence à un autre document
Array	// Tableau
Mixed	// Type quelconque

OPERATEURS DE REQUETE

\$eq	// Égal à
\$ne	// Différent de
\$gt	// Supérieur à
\$gte	// Supérieur ou égal à
\$lt	// Inférieur à
\$lte	// Inférieur ou égal à
\$in	// Dans un tableau
\$nin	// Pas dans un tableau
\$or	// OU logique
\$and	// ET logique
\$regex	// Expression régulière



BONNES PRATIQUES

- Toujours valider les données
- Utiliser les indexes pour optimiser les requêtes
- Ne jamais stocker de mots de passe en clair
- Utiliser select: false pour les données sensibles
- Gérer les erreurs de duplication
- Utiliser timestamps pour tracer les modifications
- Documenter les schémas et méthodes